Contents

Steer or Screen? How to Spend a Learned Cost-to-Go When Warm-Start Budget Is Scarce **1**
Abstract . . . 1
1. Introduction . . . 2
2. Related Work . . . 3
3. Problem Setup . . . 4
4. Methods . . . 4
5. Results . . . 5
5.1 Low-dimensional regime: nobody earns their complexity . . . 5
5.2 High-dimensional regime: the cost-to-go pays off — as a screen . . . 5
5.3 The trust region is necessary but not sufficient . . . 7
5.4 Is it the latent, or the ranker? Diffusion vs. VAE, screen vs. pick . . . 8
6. Discussion . . . 10
7. Limitations and Threats to Validity . . . 10
8. Future Work . . . 11
9. Conclusion . . . 11
References . . . 11
Reproducibility . . . 12

Steer or Screen? How to Spend a Learned Cost-to-Go When Warm-Start Budget Is Scarce

A controlled study of latent-space steering versus surrogate-guided screening for generating warm-start seeds under a matched solver-query budget.

Abstract

Learned warm-starts are known to accelerate the repeated solution of parametric nonconvex optimization problems, but most evidence concerns *convergence speed* rather than the harder question of *which basin* the solver ends up in. This paper isolates a single, sharp design question. Given a trained conditional generative model over good solutions and a learned cost-to-go predictor, and given a fixed budget of expensive solver calls, is it better to (i) **steer** — perform cheap gradient descent on the cost-to-go in the model's latent space and decode a seed — or (ii) **screen** — draw many latent samples and solve only the few the surrogate ranks best? Both use the same trained components; they differ only in how they spend the scarce budget.

We build a dimension-general family of nonconvex multi-well landscapes in which the global basin moves with an instance parameter, and we evaluate six seed-generation strategies at matched budget under two regimes (a low-dimensional, densely-covered instance space and a high-dimensional, sparsely-covered one) and two test distributions (in-distribution and shifted). The results are consistent and, for the steering hypothesis, largely negative. **In the easy regime every structured method is near-optimal and nearest-neighbor retrieval is unbeatable, so no learned method earns its complexity.** In the hard regime the learned cost-to-go *does* pay off — but the winning way to use it is **screening**, which attains a median optimality gap of zero at every

budget including a single solver call under distribution shift, whereas **latent steering never wins and degrades as budget grows** because it collapses its seeds onto a single mode and forfeits the diversity that scarce-budget multi-start depends on. We also confirm that steering without a trust region is actively harmful: the surrogate is exploited off-manifold, promising low cost it cannot deliver. Finally, we replace the generative model with a conditional diffusion sampler and screen it with an **architecture-agnostic** cost-to-go ranker defined in solution space: diffusion-plus-screen and VAE-plus-screen are statistically indistinguishable and both attain a median gap of zero, while neither sampler helps without the ranker. We conclude that in this setting the contribution is the *cost-to-go signal*, not the *latent geometry* nor the *choice of generator*; a learned ranker over samples is the honest, robust way to allocate solver calls, and gradient-based latent optimization is the wrong instrument for the job unless diversity is explicitly restored.

---

1. Introduction

Many deployed systems solve the *same* optimization problem over and over with only its parameters changing: model-predictive control re-solves a trajectory problem each timestep, a portfolio system re-optimizes as prices move, a manipulator re-plans as the scene shifts. When the objective is nonconvex, the initial point ("seed", "warm-start") strongly determines both how fast a local solver converges and, crucially, *which local minimum* it converges to. This has motivated a large body of work on learning to predict good warm-starts from problem parameters, most of it under the umbrella of **amortized optimization** [1]: pay a one-time offline training cost to learn a fast predictor, then warm-start a classical solver at inference.

The literature is clear that learned warm-starts reduce iterations and runtime relative to naive initialization [1,2,3]. It is far less clear that they reach *lower minima* — a strictly stronger claim, because it requires the learned seed to land in a *better basin*, not merely a *closer* one to whatever basin a baseline would have found. That distinction is the crux of this paper. Measuring "faster" is easy and common; measuring "better basin, at equal cost" is neither, and it is where the interesting uncertainty lives.

Two families of methods promise better basins. **Deterministic single-seed predictors** [2,3] map parameters to one warm-start with a downstream-aware loss; they are fast but a single seed still gets trapped. **Generative seed models** — recently diffusion and flow-matching models trained on locally-optimal solutions [4,5,6] — instead produce *many diverse* seeds so the solver can explore several basins. Diversity is their selling point and their cost: sampling is untargeted, so the budget is spent partly on redundant or poor seeds.

An autoencoder-style latent model suggests a third option that a pure sampler cannot offer. If the latent is smooth and we additionally learn a **cost-to-go** predictor $g(z, \theta)$ estimating the final solved cost of decoding z, then at test time we can *optimize* — run cheap gradient descent on g in latent space toward a low-predicted-cost point, decode it, and hand it to the solver. This is the same idea as latent-space optimization for molecular design [7], repurposed for solver initialization. The appeal is that expensive solver calls are replaced by cheap surrogate-gradient steps.

This paper asks exactly one question (call it RQ5): at a matched budget of expensive solver calls, does latent-space *steering* produce better seeds than *sampling-and-screening* from the same model? The question matters because it is the specific thing that would justify an autoencoder-style latent over a diffusion sampler — steering is the one capability a smooth latent adds. If steering does

not win, the honest conclusion is that the latent geometry is a sideshow and the learned cost-to-go is the real asset.

Our contribution is not a new state-of-the-art method. It is a controlled, budget-matched, distribution-shift-aware comparison that answers RQ5 with a clear and somewhat deflating result, and that identifies *why* steering underperforms. We believe the negative result is more useful than another warm-start method, because it redirects effort from latent-space optimization toward the component that actually carries the weight.

---

2. Related Work

Amortized optimization. Amos [1] gives the unifying view: learn to predict solutions to a distribution of related problems, exploiting shared structure, and optionally fine-tune the prediction by warm-starting a solver. Our setting is a special case — the predicted object is a seed, and the "fine-tuning" is the local solver — but our question is orthogonal to the framework: *how* to query a trained amortized model under a query budget.

Learned warm-starts with downstream losses. Sambharya, Hall, Amos and Stellato [2,3] learn to warm-start fixed-point algorithms end-to-end, training either a fixed-point-residual loss or a distance-to-solution loss, and provide PAC-Bayes generalization bounds. Their setting is convex, deterministic, and single-seed; "better basin" is not meaningful there. Our work is the nonconvex, multi-seed counterpart, where basin choice is the whole game and no such guarantees are available.

Decision-focused warm-starts. Briden and colleagues [8] warm-start trajectory optimization by predicting full primal–dual solutions with merit-function (decision-focused) losses, emphasizing runtime and constraint satisfaction. This is the closest prior work to a "cost-aware" learned seed, but again a single predicted point, optimized for feasibility and speed rather than for reaching lower minima across many basins.

Generative diverse seeds. DiffuSolve [4] trains a conditional diffusion model on locally-optimal solutions, samples diverse candidates, and warm-starts an NLP solver, reporting large efficiency gains and a constrained variant that penalizes constraint violation during training. DiffusionSeeder and point-cloud-conditioned flow-matching seeders [5,6] make the explicit argument that a single deterministic seed still risks local minima and that multiple diverse seeds let the optimizer find minima across different modes. These methods define the competitive landscape our "sample-and-pick" baseline stands in for: they *sample*, they do not *steer*. Our screening baseline adds the one ingredient they lack — a learned ranker over the samples.

Latent-space optimization. Gómez-Bombarelli et al. [7] popularized optimizing a learned objective over a continuous autoencoder latent. Our "latent-opt" method is that idea applied to seed generation. Our finding — that it underperforms screening and collapses diversity — is a cautionary data point for transplanting latent-space optimization into warm-starting.

The gap this paper fills is methodological rather than architectural: none of the above isolates *steering vs. screening at matched budget* under distribution shift, which is the comparison that decides whether latent geometry is worth anything here.

---

3. Problem Setup

Parametric problem family. We consider a family of objectives $f(x;\theta)$ with decision variable $x \in \mathbb{R}^D$ and instance parameter $\theta \in \mathbb{R}^P$. The landscape is a sum of K Gaussian wells at fixed centers $c_k \in \mathbb{R}^D$ plus a mild quadratic bowl:

$$f(x;\theta) = -\sum_{k=1}^{K} d_k(\theta)\, \exp\Big(-\tfrac{\|x-c_k\|^2}{2\sigma^2}\Big) + \lambda\|x\|^2, \qquad d_k(\theta) = \mathrm{softplus}(b_k + \theta^\top w_k).$$

The key property is that each well's depth $d_k(\theta)$ depends on θ, so **which well is deepest — the global basin — moves as θ varies.** A good seed must learn, for a given θ, which well to head toward. Naive seeds fall into the nearest well, which is generally not the deepest. This makes warm-starting a genuine learning problem rather than a fixed lookup.

Solver and the notion of a "query." The local solver is Adam-based gradient descent on f for a fixed number of steps from a seed x_0. A *query* is one solver call; it is the expensive operation and the only thing we ration. Every method is compared at a matched number of queries B. Cheap surrogate-gradient steps are free and are not counted.

Reference and metric. For each test instance we estimate the best attainable cost by a large random multi-start (R restarts) and call it the reference $c^\star(\theta)$. The evaluation metric is the **optimality gap**, $\mathrm{gap} = c_{\mathrm{method}} - c^\star(\theta)$, clipped at zero. A gap near zero means the method reliably reaches the global basin. We report the **median** gap (typical performance) and the **90th-percentile** gap (tail / worst-case reliability) across instances. Reporting the tail matters: a method can have an excellent median yet occasionally miss entirely.

Regimes and distribution shift. We instantiate two regimes to test the hypothesis that cheap baselines look strong only because low-dimensional instance spaces are densely covered by training data:

- **Regime A (low-dim / dense):** $D = 2$, $P = 2$, $K = 6$.
- **Regime B (high-dim / sparse):** $D = 8$, $P = 6$, $K = 12$.

For each regime we evaluate two test distributions: **in-distribution** ($\theta \sim \mathcal{N}(0, I)$, matching training) and **shifted** ($\theta \sim \mathcal{N}(1.5,\, 1.6^2 I)$, a mean-and-variance shift away from training).

---

4. Methods

All learned methods share the same trained components, so the comparison isolates *how the budget is spent*, not *what was trained*.

Generative model. A conditional VAE with a Z-dimensional latent, small MLP encoder/decoder, trained to reconstruct the best-known solutions per training instance. After training, $\mathrm{dec}(z, \theta)$ maps a latent to a candidate seed and $z \sim \mathcal{N}(0, I)$ gives samples.

Cost-to-go surrogate $g(z, \theta)$. We sample z from the prior, decode to a seed, *solve*, and record the achieved cost; we then regress $g(z, \theta) \approx$ solved cost on these (offline, uncounted) labels. This is the component that makes the latent steerable and that makes sample ranking possible.

The six strategies, all at budget B solver calls:

1. **random** — B uniform random seeds; solve all; keep best. (Multi-start baseline.)
2. **retrieval** — the B nearest-θ training solutions used as seeds. A deceptively strong classical baseline.
3. **sample-pick** — draw B latents, decode, solve all, keep best. The diffusion-style "sample diverse seeds" move, stripped to its essence.
4. **sample-screen** — draw a large pool of latents, rank by g, solve only the top-B, keep best. Sampling *plus* the learned ranker.
5. **latent-opt (trust)** — from B random latents, run gradient descent on $g(z, \theta) + \tau\|z\|^2$ (a prior/trust-region penalty), decode, solve all B, keep best. The *steering* method. Surrogate steps are free; only the B solves count.
6. **latent-opt (no trust)** — same but $\tau = 0$, included to expose surrogate exploitation.

The comparison of interest for RQ5 is **(5) vs (3)** — does steering beat blind sampling — sharpened by **(5) vs (4)** — does steering beat sampling *with the same cost signal used only as a filter*. Method (4) is the crucial control: if steering barely beats sample-pick but loses to sample-screen, then the benefit is the surrogate, not the steering.

---

5. Results

Figure 1 plots the median optimality gap versus budget for all four (regime × test-distribution) settings. Tables 1–2 give median and 90th-percentile gaps. Figure 2 shows the surrogate-exploitation phenomenon in the high-dimensional regime.

Figure 1. Median optimality gap (lower is better) vs. matched budget B. Top row: low-dim regime A; bottom row: high-dim regime B. Left column: in-distribution; right column: shifted. In A everyone but *random* sits on the floor. In B the ordering separates: *sample-screen* hugs zero everywhere, while *latent-opt (trust)* stays high and even worsens relative to sampling at larger budgets.

5.1 Low-dimensional regime: nobody earns their complexity

In regime A, at a single solver call, *random* has a median gap of 0.72 while **every structured method — retrieval, sample-pick, sample-screen, and both latent variants — has a median gap of essentially zero**, in and out of distribution. The only separation is in the tails: at B=1 in-distribution the 90th-percentile gap is 0.12 for sample-pick, 0.01 for latent-opt (trust), and 0.00 for retrieval and sample-screen. In other words, when the instance space is two-dimensional and densely covered by training data, *nearest-neighbor retrieval already solves the problem*, and no learned method — steering included — provides a median advantage. This is the first honest lesson, and it is a warning about toy benchmarks: a 2-D testbed makes retrieval look unbeatable and hides all the interesting differences.

5.2 High-dimensional regime: the cost-to-go pays off — as a screen

Regime B is where methods separate. Table 1 (in-distribution) and Table 2 (shifted) report median (p90) gaps.

Table 1. Regime B, in-distribution — median gap (p90).

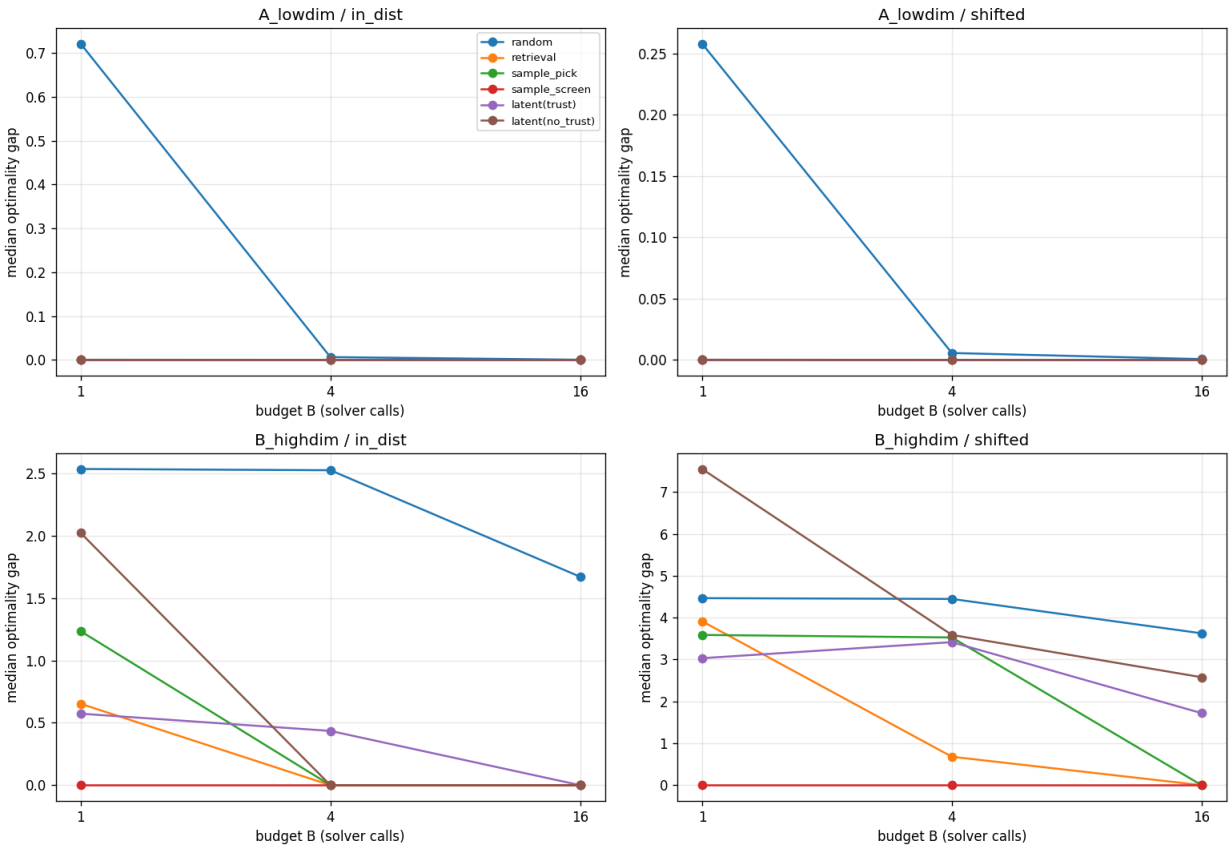


Figure 1: Figure 1: median optimality gap vs. matched solver-query budget, across two regimes and two test distributions.

method	B=1	B=4	B=16
random	2.535 (5.08)	2.525 (5.38)	1.671 (4.68)
retrieval	0.653 (3.44)	0.000 (2.70)	0.000 (0.00)
sample-pick	1.235 (4.41)	0.000 (3.35)	0.000 (1.07)
sample-screen	**0.000 (1.32)**	**0.000 (0.18)**	**0.000 (0.00)**
latent-opt (trust)	0.573 (3.02)	0.435 (3.01)	0.000 (2.58)
latent-opt (no trust)	2.023 (8.17)	0.000 (3.54)	0.000 (1.34)

Table 2. Regime B, shifted — median gap (p90).

method	B=1	B=4	B=16
random	4.467 (10.07)	4.448 (10.06)	3.627 (8.99)
retrieval	3.915 (9.09)	0.679 (8.03)	0.000 (0.77)
sample-pick	3.588 (9.79)	3.528 (9.02)	0.000 (4.39)
sample-screen	**0.000 (6.36)**	**0.000 (1.29)**	**0.000 (0.00)**
latent-opt (trust)	3.033 (8.02)	3.416 (8.02)	1.720 (7.28)
latent-opt (no trust)	7.551 (13.55)	3.588 (9.80)	2.578 (8.91)

Three facts stand out.

Screening dominates. *sample-screen* achieves a median gap of zero in every high-dimensional condition and at every budget, including a *single* solver call under distribution shift (Table 2, B=1). Using the learned cost-to-go to rank a pool of samples and solve only the best few is, by a wide margin, the most reliable way to spend a scarce budget.

Steering never wins and gets worse with budget. *latent-opt (trust)* is better than random, retrieval, and sample-pick at B=1 (0.573 in-distribution) — so steering does help relative to blind sampling — but it is dominated by *sample-screen* everywhere and, tellingly, its median gap does **not** improve from B=1 to B=4 (it goes $0.573 \to 0.435$ in-distribution and $3.033 \to 3.416$ shifted, essentially flat or worse). By B=16 under shift it is the second-worst method overall (1.720), beaten even by plain sample-pick. The mechanism is diversity collapse: steering pushes all B latents toward the same low-g region, so the B seeds are near-duplicates and multi-start buys almost nothing. Sampling-based methods, whose seeds are spread out, convert extra budget into extra basin coverage; steering cannot.

Retrieval degrades exactly where predicted. In the high-dimensional, sparsely-covered instance space, nearest-neighbor retrieval is no longer near-perfect at low budget (0.653 in-distribution, B=1), and under distribution shift it collapses (3.915 at B=1). This confirms that its apparent invincibility in regime A was an artifact of dense low-dimensional coverage, and it validates raising the instance dimension as the right stress test.

5.3 The trust region is necessary but not sufficient

latent-opt (no trust) is the worst method at B=1 in the hard regime (2.023 in-distribution, 7.551 shifted). Figure 2 shows why: with no trust penalty, gradient descent drives z into regions where the surrogate confidently predicts very low cost but the decoded seed actually solves to high cost — points far below the diagonal. The surrogate is exploited off-manifold. Adding the trust penalty

pulls the optimized latents back toward the prior where g is accurate (the green cluster near the diagonal). But even with the trust region, steering only reaches parity-to-worse against screening. So the trust region rescues steering from catastrophe without making it competitive.

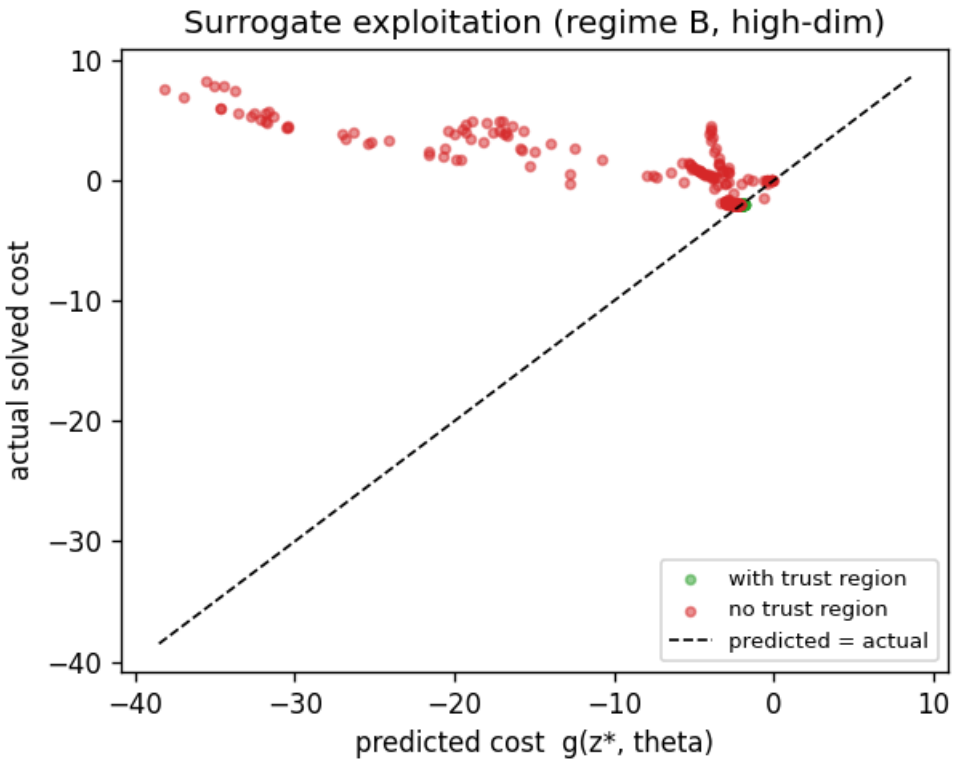


Figure 2: Figure 2: predicted vs. actual solved cost for latent optimization with and without a trust region, high-dim regime.

Figure 2. Predicted cost $g(z^\star, \theta)$ vs. actual solved cost, high-dim regime. Without a trust region (red), the surrogate promises low cost it cannot deliver — off-manifold exploitation. With a trust region (green), predictions stay honest and tightly clustered, but as Tables 1–2 show this still does not beat screening.

5.4 Is it the latent, or the ranker? Diffusion vs. VAE, screen vs. pick

The results so far show screening winning, but they leave a confound: the screening ranker was defined on the VAE latent. Perhaps the VAE latent is doing hidden work. To settle whether the asset is the *cost-to-go signal* or the *generator/latent*, we ran a follow-up in the hard regime with two changes. First, we added a **conditional diffusion (DDPM) sampler** over good solutions, the modern tool of choice for diverse seeds. Second, we defined the cost-to-go ranker $h(x_0, \theta)$ in **solution space** rather than in any model's latent — it predicts solved cost directly from a candidate seed, so the identical ranker can screen samples from *any* generator. We trained h on seeds drawn from a mixture of sources (random, VAE, diffusion) so it favors no generator.

Table 3 reports the outcome (a separate run with N_{test}=30, R=64, and the solution-space ranker, so absolute numbers differ slightly from Tables 1–2 but the ordering is identical). Figure 3 plots it.

Table 3. Regime B with an architecture-agnostic ranker — median gap (p90). Screening rows in bold.

method	in-dist B=1	in-dist B=4	in-dist B=16	shifted B=1	shifted B=4	shifted B=16
random	2.134 (6.02)	1.819 (4.60)	0.618 (3.10)	3.490 (8.99)	3.325 (8.91)	2.359 (7.44)
retrieval	0.360 (3.74)	0.000 (0.38)	0.000 (0.02)	3.136 (8.88)	0.000 (6.20)	0.000 (0.00)
vae-pick	1.165 (3.70)	0.354 (3.01)	0.000 (0.44)	3.079 (8.88)	2.229 (8.88)	0.000 (3.87)
vae-screen	**0.000 (0.33)**	**0.000 (0.02)**	**0.000 (0.33)**	**0.000 (3.21)**	**0.000 (0.00)**	**0.000 (0.00)**
diff-pick	1.703 (4.56)	0.449 (3.72)	0.000 (3.05)	3.136 (8.88)	2.726 (8.88)	1.415 (8.33)
diff-screen	**0.000 (0.35)**	**0.000 (0.74)**	**0.000 (0.44)**	**0.000 (3.56)**	**0.000 (7.37)**	**0.000 (5.73)**
latent-opt (trust)	0.000 (3.20)	0.168 (3.20)	0.000 (2.49)	3.079 (8.33)	2.345 (8.33)	2.345 (8.33)

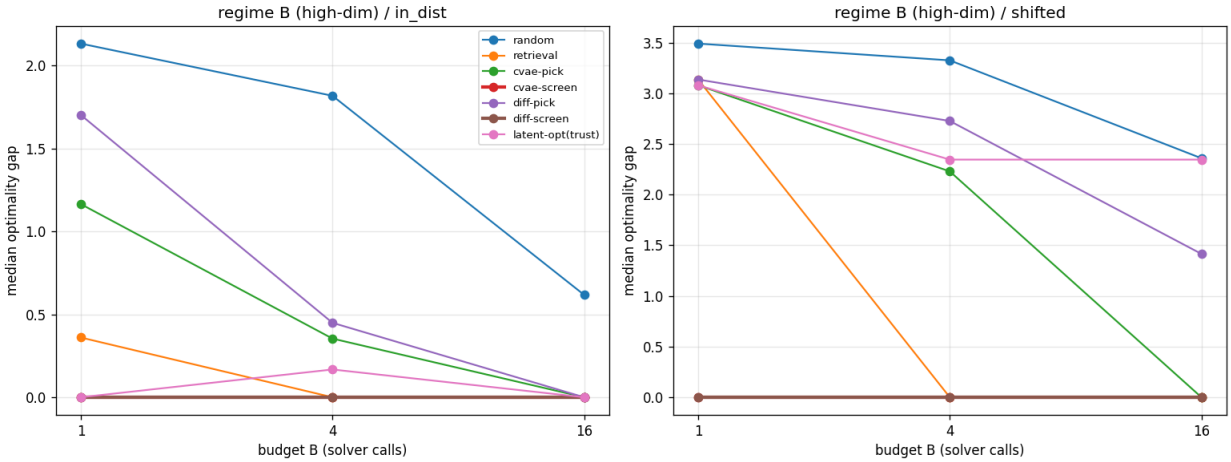


Figure 3: Figure 3: diffusion vs. VAE and screen vs. pick, high-dim regime.

Figure 3. Median gap vs. budget in the high-dim regime, in-distribution (left) and shifted (right). The two screening curves (thick) sit on the floor regardless of the generator; the two "pick" curves (thin) track each other well above it; latent steering stays high under shift.

The verdict is unambiguous. **Screening attains a median gap of zero at every budget and under shift regardless of which generator feeds it** — vae-screen and diff-screen are indistinguishable at the median (the VAE has slightly tighter tails under shift). **Neither generator helps without the ranker:** vae-pick and diff-pick are both mediocre at low budget and track each other, so the diffusion model's celebrated sample diversity buys nothing here once the budget is scarce and the seeds are unranked. And because the winning ranker lives in solution space, its benefit is *portable across generators* — direct evidence that the useful object is the learned cost-to-go, not the latent geometry or the sampler. Latent steering again fails to help and stays high under distribution shift.

6. Discussion

The headline answer to RQ5 is negative for the exciting hypothesis and positive for a humbler one. **Latent-space steering does not beat sampling-and-screening at matched budget; it never wins in the regime where the choice matters, and it degrades as budget grows because it destroys seed diversity.** What *does* carry the weight is the learned cost-to-go — but the right way to use it is as a *ranker over diverse samples*, not as a *gradient to optimize a single mode toward*.

This reframes where the value lives. The smooth, searchable latent — the one thing an autoencoder offers over a diffusion sampler — turns out to be a liability here, because the natural way to exploit it (gradient descent) collapses diversity, and diversity is precisely what scarce-budget multi-start needs. Meanwhile the cost signal is architecture-agnostic, and §5.4 confirms this directly: a solution-space ranker screens diffusion and VAE samples equally well, and neither generator is useful without it. So the case for an autoencoder-specific pipeline — or for any particular generator — is empirically weak; "any competent sampler + a learned cost-to-go ranker" captures the benefit, and the ranker is where the effort should go. If one insists on steering, it must be *diversified* steering — steering toward several distinct low-cost modes, e.g. with a repulsion term or quality-diversity machinery — which is future work, not a result here.

The findings also caution against toy benchmarks. In two dimensions, retrieval and every learned method are indistinguishable at the median; only in higher-dimensional, sparsely-covered, shifted regimes do methods separate. Any warm-start study that reports only low-dimensional in-distribution results is likely measuring the benchmark, not the method.

---

7. Limitations and Threats to Validity

This is a controlled study on a synthetic family, and its conclusions should be read at that scope.

- **Synthetic landscapes.** Gaussian-well objectives are a clean, controllable stand-in for nonconvex structure, but real trajectory-optimization or MPC landscapes have constraints, higher dimension, and correlated basins we do not model. The qualitative ordering may change on real problems.
- **Modest instance dimension.** $P = 6$ is "high" only relative to $P = 2$. Retrieval degradation and steering's diversity collapse should be re-checked at substantially higher P.
- **Generator scope.** §5.4 adds a conditional diffusion (DDPM) sampler and a shared solution-space ranker, which is enough to show the ranker's benefit is generator-independent. But our DDPM is small and trained on a modest solution set; a larger, better-tuned diffusion or flow-matching model on real problems might change the pick-vs-pick comparison even if, we expect, not the screen-dominates conclusion.
- **Surrogate quality is a confound.** Screening's dominance depends on g being an accurate ranker; on problems where cost-to-go is hard to predict, screening could fail and the picture could change. We did not stress-test g accuracy.
- **No theory.** We offer a mechanism (diversity collapse) supported by the budget-scaling pattern, not a proof. A formal account of when steering must lose to screening would strengthen the claim.
- **Single seed, small nets, one solver.** Results are averaged over instances but not over training seeds or architectures; error bars over training randomness are not reported.

None of these undermine the internal comparison — steering, screening, and sampling share every component and every budget — but they bound external validity.

---

8. Future Work

The most decisive next experiments follow directly. First, **diversified steering**: add a repulsion or quality-diversity term so steering targets several distinct low-cost modes, and test whether that recovers the budget-scaling that plain steering loses — this is the one route by which latent geometry might still earn its place against screening. Second, **push P higher and add real constraints** (a genuine trajectory-optimization or MPC family) to see whether screening's reliability and retrieval's collapse persist beyond synthetic wells. Third, **stress-test the ranker**: characterize how screening degrades as the cost-to-go's ranking accuracy drops, since that accuracy is the assumption the whole result rests on; a poor ranker should erode screening's dominance. Fourth, a **theory-lite account** of diversity collapse — e.g., bounding the effective number of distinct basins reached as a function of the steering strength — would convert the empirical mechanism into a principle. Fifth, scale the **diffusion/flow sampler** and re-examine the pick-vs-pick comparison, now that screen-vs-pick is settled.

---

9. Conclusion

We set out to test whether optimizing a learned cost-to-go over a generative model's latent produces better warm-start seeds than sampling and screening from the same model, at equal solver-query budget. Across a dimension-general nonconvex family, two regimes, and a distribution shift, the answer is that **steering is the wrong instrument**: it helps over blind sampling but loses to surrogate-guided screening everywhere it matters, and it worsens with budget because it collapses the seed diversity that scarce-budget multi-start relies on. The asset that actually delivers is the learned cost-to-go, used as a ranker — an architecture-agnostic component that sharpens a diffusion sampler and a VAE equally, and that neither generator can do without. The practical recommendation is therefore to invest in a good cost-to-go predictor and spend the query budget on the samples it ranks best, and to treat latent-space steering as promising only if it is explicitly diversified. The negative result is the point: it redirects effort from latent geometry, which does not carry its weight here, to the cost signal, which does.

References

[1] B. Amos. *Tutorial on Amortized Optimization.* Foundations and Trends in Machine Learning, 16(5), 2023. arXiv:2202.00665.

[2] R. Sambharya, G. Hall, B. Amos, B. Stellato. *Learning to Warm-Start Fixed-Point Optimization Algorithms.* Journal of Machine Learning Research, 25(166):1–46, 2024. arXiv:2309.07835.

[3] R. Sambharya, G. Hall, B. Amos, B. Stellato. *End-to-End Learning to Warm-Start for Real-Time Quadratic Optimization.* Learning for Dynamics and Control (L4DC), 2023. arXiv:2212.08260.

[4] DiffuSolve: *Diffusion-based Solver for Non-convex Trajectory Optimization.* arXiv:2403.05571, 2024.

[5] DiffusionSeeder: generative multi-seed initialization for trajectory optimization, 2024 (as surveyed in [6]).

[6] *Warm-Starting Optimization-Based Motion Planning for Robotic Manipulators via Point Cloud-Conditioned Flow Matching.* arXiv:2510.03460, 2025.

[7] R. Gómez-Bombarelli et al. *Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules.* ACS Central Science, 4(2):268–276, 2018.

[8] J. Briden et al. *Constraint-Informed Learning for Warm-Starting Trajectory Optimization* (Trajectory Optimization with Merit-Function Warm Starts). Journal of Guidance, Control, and Dynamics, 2025.

Reproducibility

All results come from a single self-contained script (`extended_experiment.py`) and its per-regime runner. The parametric landscape, Adam-based solver with a global query counter, CVAE, cost-to-go surrogate, and the six strategies are defined there; the evaluation loop computes the reference by large multi-start and reports median/p90 gaps at matched budget. Two regimes (A: D=2, P=2, K=6; B: D=8, P=6, K=12) and two test distributions were run; figures are regenerated from the saved `paper_results.json`. The §5.4 diffusion comparison lives in `diffusion_experiment.py` (conditional DDPM sampler plus a solution-space ranker h), producing `diffusion_results.json`. The companion notebook `RQ5_latent_opt_vs_sample_pick.ipynb` contains a fully annotated single-regime version suitable for Colab.